ComfyUI LoRA Training Guide

for Sprite & Animation Style Transfer

A comprehensive guide for training custom LoRA models
for 2D pixel art and retro game sprite generation

OPTIMIZED FOR

Alienware Aurora | RTX 5080 16GB | Windows 11

Prepared for Sean Winslow — 16BitFit Project

March 2026

Table of Contents

Section 1: Training Framework Comparison

Four major open-source frameworks support LoRA training for Stable Diffusion models. Each has different strengths depending on your target model, VRAM budget, and experience level.

kohya_ss / sd-scripts (Recommended Primary)

kohya_ss (GUI maintained by bmaltais[1], powered by **sd-scripts v0.9.1**[2]) is the industry standard for SDXL LoRA training. CivitAI's entire training ecosystem is built around it, giving it the largest community and most extensive documentation of any training framework.

For SDXL, standard training requires ~24GB VRAM, but with the **fused backward pass** enabled in bf16 mode, VRAM drops to approximately **10GB** — well within your RTX 5080's 16GB. On Windows, the RTX 5080 requires the **dev branch**, **CUDA 12.9.0**, and **cuDNN 9.10.1**. You must also comment out DeepSpeed from requirements.txt. A video walkthrough for RTX 5000-series setup is available[3].

Key features include LoRA+, block-wise training, alpha mask, optimizer groups, scheduled Huber loss, and V-parameterization for SDXL. The main caveat is that **bitsandbytes** on Windows has historically caused CUDA errors — use the latest version (0.44.0+) or switch to Adafactor/AdamW if issues arise[4].

ai-toolkit by Ostris (Best for Flux)

ai-toolkit[5] (no versioned releases; use latest GitHub commit) is the go-to tool for **Flux.1** LoRA training. It supports FLUX.1 (dev, schnell, Kontext), SDXL, OmniGen2, and Wan I2V video models.

Flux training normally requires 24GB+ VRAM, but with quantization: int8 ~18GB, int4 ~13GB, **NF4 ~9GB**. Windows is supported with an easy install script. Note: Flux requires ~50GB system RAM for model quantization at startup.

OneTrainer (Best for Beginners)

OneTrainer[6] is the most beginner-friendly option with a polished GUI, templates (including an SDXL LoRA template), and integrated dataset tools. Windows is fully supported with install.bat, start-ui.bat, and update.bat. Built-in features include image augmentation, TensorBoard integration, and auto-captioning (BLIP/BLIP2/WD-1.4)[7].

SimpleTuner (Advanced / Multi-Model)

SimpleTuner v3.0.1[8] (Oct 2025) has the broadest model support: SDXL, Flux.1, SD 3, Chroma, AuraFlow, PixArt Sigma, Sana, and many video models. It is primarily Linux-focused with unclear Windows support, making it best for experienced users.

Framework Comparison

Feature	kohya_ss	ai-toolkit	SimpleTuner	OneTrainer
Best for	SDXL LoRA	Flux LoRA	Multi-model	Beginners
SDXL on 16GB	Yes (~10GB)	Less documented	Yes	Yes

Feature	kohya_ss	ai-toolkit	SimpleTuner	OneTrainer
Flux on 16GB	No	Yes (NF4 ~9GB)	Possible	Yes
Windows	Full (dev branch)	Full	Unclear	Full
GUI	Yes	Web UI	No	Best GUI
Community	Largest	Growing	Moderate	Active

---

[1] https://github.com/bmaltais/kohya_ss
[2] https://github.com/kohya-ss/sd-scripts
[3] https://youtube.com/watch?v=3IPc3dmxD54
[4] https://reddit.com/r/StableDiffusion/comments/1kotq5v/
[5] https://github.com/ostris/ai-toolkit
[6] https://github.com/Nerogar/OneTrainer
[7] https://reddit.com/r/StableDiffusion/comments/1gvp073/
[8] https://github.com/bghira/SimpleTuner

TL;DR — For Your RTX 5080 Setup

Use **kohya_ss** (bmaltais GUI, dev branch) as your primary SDXL training tool. With fused backward pass + bf16, training fits in ~10GB of your 16GB VRAM. Keep **OneTrainer** as a beginner-friendly backup. For Flux later, use **ai-toolkit** with NF4 quantization (~9GB).

Section 2: Base Model Selection

LoRA training adapts a base model toward your target style. Starting from a base that already excels at 2D illustration means fewer training steps, better results, and less overfitting risk.

Illustrious XL v0.1 (Recommended)

Illustrious XL (by OnomaAIResearch, v0.1 to v2.0) is a purpose-built SDXL finetune for anime and illustration output[1]. It delivers cleaner line work, better color reproduction, and stronger prompt adherence than base SDXL 1.0. The model "soaks concepts, art styles and characters like a sponge, usually around 500 steps gives great results."

Built on the 3.5-billion-parameter SDXL architecture, it requires only ~6–8GB VRAM for inference. **v0.1** is the most commonly used version as a LoRA training base (HuggingFace: OnomaAIResearch/Illustrious-xl-early-release-v0). LoRAs trained on Illustrious also work on **NoobAI XL** (an Illustrious finetune), giving you cross-compatibility.

Alternative SDXL Finetunes

Model	Strengths	Notes
Pony Diffusion V6 XL	Anime styles, character-centric, extensive CivitAI ecosystem	Requires clip_skip=2, >3,600 steps for styles
NoobAI XL	Broader stylistic range, improved composition	Cross-compatible with Illustrious LoRAs
Animagine XL 3.1	Open-source anime model, popular on CivitAI	Less commonly used as training base

Flux.1 Dev (Future Option)

Flux.1 Dev is a 12-billion-parameter model using flow matching architecture[2]. It offers superior text rendering and better anatomy but requires 24GB VRAM standard or ~9GB with NF4 quantization. **Flux 2 Klein 9B** is a smaller variant worth monitoring — community reports training on RTX 3060 (11GB)[3]. The ComfyUI ControlNet/IP-Adapter ecosystem for Flux is less mature than SDXL.

Models NOT Recommended for This Project

SDXL 1.0 base: Too generic for illustration — always use a finetune. **SD 3.5**: Limited community adoption for 2D illustration. **SD 1.5**: Older architecture, lower resolution, shrinking community.

[1] https://civitai.com/articles/1716/opinionated-guide-to-all-lora-training-2025-update
[2] https://pxz.ai/blog/flux-vs-sdxl
[3] https://mindstudio.ai/blog/what-is-flux-2-dev-lora/

TL;DR — For Your RTX 5080 Setup

Start with **Illustrious XL v0.1** as your base model. Its flat illustration style aligns perfectly with 2D game art and requires fewer training steps. Download from HuggingFace (OnomaAIResearch/Illustrious-xl-early-release-v0). For Flux later, monitor **Flux 2 Klein 9B**.

Section 3: Dataset Preparation

Image Count and Quality

For style LoRAs, the recommended dataset size is **30–50 images**, with 50–100 being ideal for complex styles[1]. A minimum of 15–20 images can work for simple subjects. More important than quantity is **diversity of poses, compositions, and subjects** within a consistent style.

Resolution and Aspect Ratio Bucketing

For SDXL, images should be at minimum **1024x1024**, ideally up to 2048x2048. Training frameworks automatically group images into resolution "buckets" with **bucket_reso_steps=64**. No manual square cropping needed. Enable **bucket_no_upscale** to prevent small image upscaling, and **cache_latents_to_disk** to save VRAM.

Captioning Strategy for Style LoRAs

Choose a unique **trigger word** like `16bitfit_style`. Use this caption formula[2]:

```
[trigger_word], [medium/style keywords], [content description]
```

Example: `16bitfit_style, pixel art, retro game boy style, a warrior character standing in idle pose`

Key principle: Caption everything that is NOT part of the style. Describe subjects, poses, and backgrounds — the model learns the style from what remains unspecified. **Turn off text encoder training** for style LoRAs (set text_encoder_lr to 0) to force UNet-only style learning.

Auto-Captioning Tools

Tool	Type	VRAM	Best For
JoyCaption	Natural language	~17GB (4-bit for smaller GPUs)	Flux, detailed descriptions
WD Tagger (WD-1.4)	Tag-based	Low	SDXL/Illustrious, fast tagging
Florence-2	General tagging	Low	High-speed general tags
BLIP/BLIP2	Natural language	Moderate	Built into OneTrainer

For SDXL/Illustrious, **WD Tagger** with tag-style captions works well. For Flux later, switch to **JoyCaption**[3] for natural language captions.

Regularization Images

For style LoRAs: **generally not needed**. They help prevent concept "leaking" for subject/character LoRAs, but for styles they add complexity without much benefit.

Folder Structure (kohya_ss)

```
training_data/
  10_16bitfit_style/
    image001.png
    image001.txt
    image002.png
    image002.txt
```

Use **PNG format** (mandatory for pixel art). The number prefix controls repeats per epoch. Formula: total_steps = (num_images × repeats × epochs) / batch_size.

Pixel Art and Flat 2D Illustration Tips

- **Do NOT upscale pixel art with standard upscalers** — use nearest-neighbor only[4]
- Include style keywords: "pixel art", "limited color palette", "clean outlines", "flat shading", "game boy aesthetic"
- Describe what makes your style unique: color count, line thickness, shading approach
- Do not mix pixel art with non-pixel-art images — style consistency is critical
- Crop characters on clean/transparent backgrounds where possible

---

[1] https://reddit.com/r/StableDiffusion/comments/13dh7ql/
[2] https://sanj.dev/post/lora-training-2025-ultimate-guide
[3] https://github.com/fpgaminer/joycaption
[4] https://dev.to/gary_yan_86eb77d35e0070f5

TL;DR — For Your RTX 5080 Setup
Gather 30–50 PNG images of your 16BitFit art style at 1024px+. Use nearest-neighbor upscaling only. Caption each with `16bitfit_style, pixel art, retro game boy style, [description]`. Use **WD Tagger** for auto-captioning, prepend your trigger word. Organize as `training_data/10_16bitfit_style/`. Skip regularization images.

Section 4: Training Configuration

These settings are optimized for your **RTX 5080 (16GB VRAM)** running SDXL LoRA training with **kohya_ss**[1].

Core Settings

Parameter	Value	Why
Batch size	1	16GB requires batch 1
Gradient accumulation	2	Simulates effective batch size of 2
Gradient checkpointing	Enabled	Required to fit in 16GB
Mixed precision	bf16	RTX 5080 supports bf16 natively
Fused backward pass	Enabled	Reduces VRAM from ~24GB to ~10GB
Cache latents to disk	Enabled	Saves VRAM during training

Network Architecture

Parameter	Value	Notes
LoRA rank (network_dim)	32	Sweet spot for style LoRAs (~20–40MB)
Network alpha	32	Same as rank for 1.0 scaling
Network type	networks.lora	Standard LoRA implementation

Optimizer Selection

Optimizer	Learning Rate	Best For	Notes
Adafactor (recommended)	0.0005	16GB VRAM	Required for fused backward pass; memory efficient
Prodigy	1.0 (auto)	Beginners	Auto LR, hard to overfit; slower convergence
AdamW8bit	0.0005 (UNet)	Experienced	Reliable but requires manual LR tuning

Recommended for your setup: **Adafactor** with fused backward pass — the only combination that gets SDXL training down to ~10GB VRAM.

Training Duration (for 30–50 images)

Parameter	Value
Repeats	10–15
Epochs	10

Parameter	Value
Total steps	~1,500–3,000
Expected wall time	30–90 minutes on RTX 5080
LR scheduler	cosine or cosine_with_restarts
Min SNR gamma	5
Noise offset	0.03 (start low for pixel art)

Text Encoder Settings (Style LoRAs)

Set **text_encoder_lr = 0** and **network_train_unet_only = True** to train only the UNet. Set **clip_skip = 1** for Illustrious base or 2 for Pony base.

Network Rank Guide

Rank	File Size	Best For	Risk
16	~10–20MB	Simple flat styles	May miss nuance
32 (recommended)	~20–40MB	Most style LoRAs	Balanced
64	~40–80MB	Complex styles	Overfitting with small datasets
128	~80–150MB	Very complex adaptations	Overkill for most

VRAM Troubleshooting Ladder

If you hit VRAM errors, apply these fixes in order:

1. Enable **fused_backward_pass** + **Adafactor** (~10GB)
2. Reduce resolution to 768x768
3. Reduce network_dim from 32 to 16
4. Enable **cache_latents_to_disk**
5. Last resort: Use **optimizer_groups=8** (~14GB)

Validation During Training

Set **sample_every_n_epochs = 5** and **save_every_n_epochs = 5** to generate preview images and save checkpoints. Create a sample_prompts.txt with 2–3 test prompts including your trigger word. After training, use the **X/Y/Z plot** node in ComfyUI to compare epoch checkpoints vs. LoRA strength. Watch training loss: should decrease gradually (~0.08–0.12 typical for SDXL). Plateaus = LR too low; spikes = LR too high.

[1] https://reddit.com/r/StableDiffusion/comments/1r2mzni/

TL;DR — For Your RTX 5080 Setup

Use **Adafactor** with **fused backward pass** and **bf16** — fits SDXL in ~10GB. Rank 32, alpha 32, batch 1, grad accum 2, cosine scheduler, 10 epochs with 10–15 repeats. Train UNet only (text_encoder_lr=0). Save checkpoints every 5 epochs. Time: 30–90 minutes.

Section 5: ComfyUI Integration

Loading Your Trained LoRA

Place your trained .safetensors file in `ComfyUI/models/loras/` and restart ComfyUI or refresh the model list.

Basic LoRA Workflow

Load Checkpoint (Illustrious XL) **Load LoRA** (connect MODEL and CLIP) **CLIP Text Encode** (positive + negative) **KSampler** **VAE Decode** **Save Image**.

LoRA Weight Ranges

Strength	Effect
0.5–0.7	Subtle style influence, blends with base model
0.7–0.9	Recommended starting range for style LoRAs
0.9–1.0	Strong style, may override base model characteristics
1.0+	Very strong — test carefully, can cause artifacts

Start at **0.8** and adjust based on results.

Full Sprite Pipeline: LoRA + ControlNet + IP-Adapter

This workflow combines your style LoRA with pose control and character reference consistency[1].

Required Custom Nodes:

- **ComfyUI Manager** — Essential for installing/updating all other nodes
- **ComfyUI_IPAdapter_plus**[2] — Maintenance mode since Apr 2025 but functional
- **ComfyUI ControlNet Auxiliary Preprocessors** (comfyui_controlnet_aux)
- **ComfyUI_Comfyroll_CustomNodes**[3] — LoRA stacks, multi-ControlNet

Required Models:

- ControlNet: SDXL-compatible OpenPose or depth map model
- IP-Adapter: **ip-adapter-plus_sdxl_vit-h.bin**
- CLIP Vision: **CLIP-ViT-H-14-laion2B-s32B-b79K.safetensors**

Workflow Diagram:

```
[Load Checkpoint] → Illustrious XL
    ↓
[Load LoRA] → Style LoRA (strength 0.7–0.9)
    ↓
[IPAdapter Model Loader] + [Load CLIP Vision] + [Load Image (ref)]
    ↓
[IPAdapter Advanced] → Modified model with reference style
    ↓
```

```
[ControlNet Loader] + [Load Image (pose)]
    ↓
[Apply ControlNet] ⍰ Pose control applied
    ↓
[CLIP Text Encode] ⍰ Positive + Negative prompts
    ↓
[KSampler] ⍰ [VAE Decode] ⍰ [Save Image]
```

Balancing Three Controls

Control	Range	If Too Strong	If Too Weak
ControlNet	0.5–0.8	Output too rigid	Pose drifts
IP-Adapter	0.5–0.8	Over-copies reference	Character features drift
LoRA	0.7–0.9	Artifacts / overtrained	Style inconsistent

Advanced: LoRA Masking and Scheduling

ComfyUI's December 2024 update added native LoRA scheduling[4]. Use **Create Hook LoRA** for region-specific application and **Hook Keyframes** to schedule strength across sampling steps.

Additional Useful Nodes:

- **Efficiency Nodes**: KSampler (Efficient) for better performance
- **ComfyUI_VNCCS**: Sprite Generator node for automating sprite sheet creation

---

[1] https://comfyui.org/en/image-style-transfer-controlnet-ipadapter-workflow
[2] https://github.com/cubiq/ComfyUI_IPAdapter_plus
[3] https://github.com/Suzie1/ComfyUI_Comfyroll_CustomNodes
[4] https://blog.comfy.org/p/masking-and-scheduling-lora-and-model-weights

TL;DR — For Your RTX 5080 Setup
Place .safetensors in ComfyUI/models/loras/. Start with Load Checkpoint Load LoRA KSampler at strength 0.8. For sprites, install **IPAdapter_plus**, **ControlNet Preprocessors**, and **Comfyroll Nodes**. Balance ControlNet (0.5–0.8), IP-Adapter (0.5–0.8), LoRA (0.7–0.9).

Section 6: Automated Sprite Optimization

The Autoresearch Concept

Andrej Karpathy's **autoresearch** pattern[1] describes an AI agent that autonomously edits parameters, runs fixed-budget experiments, evaluates against a metric, keeps improvements, and discards failures. You can adapt this for your ComfyUI sprite pipeline, running parameter sweeps overnight on your RTX 5080.

Parameters to Optimize

Parameter	Search Range
ControlNet strength	0.3–1.0
IP-Adapter weight	0.3–1.0
LoRA strength	0.5–1.2
CFG scale	3–12
Sampler	euler, euler_ancestral, dpmpp_2m, dpmpp_2m_sde
Steps	15–40
Denoising strength	Variable

Sprite Consistency Metrics

Metric	What It Measures	Library	Best For
LPIPS (primary)	Perceptual similarity	lpips	Style consistency
CLIP Similarity (primary)	Semantic similarity	transformers	Character identity
SSIM (secondary)	Structural similarity	skimage.metrics	Pixel-level checks
Subject Consistency	Cross-frame identity	vbench	Character across poses

Use **LPIPS**[2] + **CLIP Similarity** as primary metrics. Optionally use a vision LLM to grade batches 1–10 for "looks like same artist/character."

ComfyUI API for Batch Generation

ComfyUI exposes a REST API + WebSocket interface[3]:

```
POST http://127.0.0.1:8188/prompt (queue generation)
GET http://127.0.0.1:8188/history/prompt_id (get results)
GET http://127.0.0.1:8188/view?filename=... (download images)
```

Export your workflow as API format JSON (Enable Dev Mode Save API Format), load in Python, modify parameters programmatically, submit, wait via WebSocket, download and evaluate.

Building the Optimization Loop

```
1. Define search space (parameter ranges)
2. Write evaluation function (LPIPS + CLIP similarity)
3. Generate batches via ComfyUI API
4. Score each batch
5. Bayesian optimization (optuna) or grid search
6. Log results to CSV/JSON
7. Run overnight (~2-5 sec/image at 1024x1024)
```

Libraries needed: **requests**, **lpips**, **torch**, **transformers** (CLIP), **scikit-image** (SSIM), **optuna** (Bayesian optimization).

Existing Resources

- **Sprite Sheet Diffusion**[4] — Academic paper with SSIM/PSNR/LPIPS evaluation scripts
- **ComfyUI_VNCCS** — Sprite Generator node for ComfyUI
- **2D Pixel Toolkit** and **Sprite Sheet Maker** — CivitAI LoRAs for pixel art sprites

---

[1] https://github.com/karpathy/autoresearch
[2] https://github.com/richzhang/PerceptualSimilarity
[3] https://cohorte.co/blog/the-comfyui-production-playbook
[4] https://arxiv.org/html/2412.03685v2

> **TL;DR — For Your RTX 5080 Setup**
> Use ComfyUI's API to batch-generate sprites with different parameters. Score with **LPIPS** + **CLIP Similarity**. Use **optuna** for Bayesian optimization. Your RTX 5080 does ~2–5 sec/image at 1024x1024 — hundreds of combinations overnight.

Section 7: Common Pitfalls & Troubleshooting

Style LoRA Failure Modes

Problem	Likely Cause	Fix
Style doesn't transfer	Too few steps, LR too low, or text encoder stealing UNet capacity	Increase steps, raise LR, text_encoder_lr=0
Characters look like training data	Overfitting	Reduce epochs, lower rank, more dataset diversity
Colors/palettes wrong	Base model bias	Add color descriptions to captions, adjust noise_offset
Blurry/muddy output	Mixed quality in dataset	Ensure all images are crisp and high-res
LoRA only works at 1.0	Undertrained	Increase steps or learning rate
LoRA breaks above 0.5	Overtrained	Use an earlier epoch checkpoint

Overfitting: Signs and Prevention

Signs: Generated images reproduce exact poses from training data. Artifacts or color banding appear. LoRA only works at specific strengths. Diversity decreases despite varied prompts.

Prevention: Use **Prodigy** optimizer (auto-adjusts LR). Save checkpoints every 5–10 epochs. Use X/Y/Z plotting. Keep **network_dim at 32 or lower**. Ensure dataset diversity.

Underfitting: Signs and Fixes

Signs: Output barely differs from base model. Trigger word has minimal effect.

Fixes: Double training steps. Increase LR (5e-4 1e-3; Prodigy: try 1.5). Increase network_dim (16 32). Verify captions and file paths.

VRAM Optimization Checklist for RTX 5080

- **Fused backward pass** (Adafactor only): ~10GB for SDXL
- **cache_latents_to_disk**: Offloads latent cache to SSD
- **Gradient checkpointing**: Trades compute for memory
- **bf16 mixed precision**: Native on RTX 5080
- **batch_size=1** with gradient_accumulation_steps=2
- **xformers or torch SDPA**: Memory-efficient attention
- **Close other GPU applications** during training

Windows-Specific Issues

1. bitsandbytes CUDA errors: The #1 Windows issue. Use latest version (0.44.0+). If errors persist, switch to **Adafactor** or standard AdamW (not 8-bit).

2. RTX 5000 series: Requires **CUDA 12.9+**, **cuDNN 9.10.1+**, **PyTorch 2.5+**. Use kohya_ss **dev branch**. Comment out DeepSpeed from requirements.txt.

3. Path length: Windows has 260-character limit. Keep training paths short.

4. CUDA version check:

```
python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda)"
```

5. Memory leaks: Monitor with nvidia-smi. Restart if VRAM climbs unexpectedly.

6. Antivirus: Add ComfyUI and training directories to Windows Defender exclusion list.

> **TL;DR — For Your RTX 5080 Setup**
> Most common issues: **bitsandbytes CUDA errors** (use Adafactor), **RTX 5080 dev branch + CUDA 12.9** (follow video guide), and **overfitting** (checkpoints every 5 epochs, X/Y/Z plots). Verify CUDA setup first. Add training folders to Defender exclusions.

Getting Started Checklist

Follow these steps in order to go from zero to your first trained LoRA and sprite pipeline.

Getting Started Checklist

1. **Install prerequisites**: CUDA 12.9.0, cuDNN 9.10.1, Python 3.10–3.12, Git, VS C++ Redistributable
2. **Clone kohya_ss** from github.com/bmaltais/kohya_ss — switch to dev branch
3. **Set up environment**: Comment out DeepSpeed, run setup.bat, configure Accelerate for bf16
4. **Download Illustrious XL v0.1** from HuggingFace (OnomaAIResearch/Illustrious-xl-early-release-v0)
5. **Prepare dataset**: 30–50 PNG images of your art style at 1024px+
6. **Caption images**: WD Tagger or JoyCaption, prepend trigger word (`16bitfit_style`)
7. **Organize folder**: `training_data/10_16bitfit_style/` with image + .txt pairs
8. **Configure training**: Adafactor, fused backward pass, bf16, batch 1, grad accum 2, rank 32, alpha 32, LR 0.0005 (UNet only), cosine scheduler, 10 epochs, sample every 5
9. **Start training**: Monitor loss, check previews (~30–90 min)
10. **Test in ComfyUI**: Load Checkpoint Load LoRA (strength 0.8) generate with trigger word
11. **Find sweet spot**: X/Y/Z plot varying epoch checkpoints vs. LoRA strength
12. **Build sprite pipeline**: Add ControlNet (OpenPose) + IP-Adapter
13. **Automate**: Build autoresearch loop with ComfyUI API + LPIPS/CLIP metrics

Key Resources

- **kohya_ss GUI**: https://github.com/bmaltais/kohya_ss
- **sd-scripts**: https://github.com/kohya-ss/sd-scripts
- **ai-toolkit**: https://github.com/ostris/ai-toolkit
- **OneTrainer**: https://github.com/Nerogar/OneTrainer
- **JoyCaption**: https://github.com/fpgaminer/joycaption
- **ComfyUI_IPAdapter_plus**: https://github.com/cubiq/ComfyUI_IPAdapter_plus
- **Comfyroll Custom Nodes**: https://github.com/Suzie1/ComfyUI_Comfyroll_CustomNodes
- **LoRA Training 2025 Guide**: https://sanj.dev/post/lora-training-2025-ultimate-guide
- **CivitAI LoRA Training Guide**: https://civitai.com/articles/1716/opinionated-guide-to-all-lora-training-2025-update
- **Cohorte ComfyUI Playbook**: https://cohorte.co/blog/the-comfyui-production-playbook
- **Karpathy Autoresearch**: https://github.com/karpathy/autoresearch
- **LPIPS Library**: https://github.com/richzhang/PerceptualSimilarity
- **Sprite Sheet Diffusion Paper**: https://arxiv.org/html/2412.03685v2
- **RTX 5090 kohya Install Video**: https://youtube.com/watch?v=3lPc3dmxD54